

Reglas de Inferencia

Nicolás Andrés Cagliero, Mauricio Jeremías Llugdar y Gregorio Vilardo

Facultad de Matemática, Astronomía, Física y Computación
Universidad Nacional de Córdoba
{nicolas.cagliero, mauriciojllugdar, gregovilardo}@mi.unc.edu.ar

1. Introducción

Las *reglas de inferencia* (inference rules) se utilizan para especificar qué *juicios de tipo* (type judgements) son válidos. Antes de empezar, asumiendo que el lector acaba de terminar con la materia Lenguajes y Compiladores de la FaMAF, intentaremos acortar la brecha de desconocimiento que posee, o mejor dicho, la brecha de conocimiento que le falta, introduciendo lo siguiente[1]:

$$\begin{aligned} \langle type \rangle ::= & int \mid bool \mid \langle type \rangle \rightarrow \langle type \rangle \\ & | prod(\langle type \rangle, \dots, \langle type \rangle) \mid sum(\langle type \rangle, \dots, \langle type \rangle) \\ & | list \langle type \rangle \mid cont \langle type \rangle \end{aligned}$$

- El **tipo primitivo** `int` denota el conjunto de los números enteros.
- El **tipo primitivo** `bool` denota el conjunto de los valores booleanos.
- El **tipo función** $\theta_0 \rightarrow \theta_1$ denota el conjunto de funciones que devuelven un valor de tipo θ_1 cuando se aplican a un valor de tipo θ_0 .
- El **producto** $prod(\theta_0, \dots, \theta_{n-1})$ denota el conjunto de tuplas con n campos de tipos $\theta_0, \dots, \theta_{n-1}$.
- La **suma** o **unión disjunta** $sum(\theta_0, \dots, \theta_{n-1})$ denota el conjunto de valores alternativos con etiquetas menores que n , donde un valor etiquetado con k tiene tipo θ_k .
- **list** θ denota el conjunto de listas cuyos elementos tienen tipo θ .

Con las siguientes abreviaciones:

$$\begin{aligned} \theta_0 \times \theta_1 &\stackrel{\text{def}}{=} prod(\theta_0, \theta_1) \\ \text{unit} &\stackrel{\text{def}}{=} prod() \\ \theta_0 + \theta_1 &\stackrel{\text{def}}{=} sum(\theta_0, \theta_1). \end{aligned}$$

donde **unit** nos permite de alguna forma definir el famoso tipo *void* de algunos lenguajes de programación, esto es útil porque cualquier demostración sobre `prod` con tipos también sirve para el "tipo (producto) vacío".

Para hacer un equivalente a un lenguaje de programación actual como Rust, podemos ver que la **suma** sería un *enum*, y el **producto** sería una *tuple* de tipos.

```

// Producto (con elementos) y Producto Vac o (unit) como
// retorno
let mi_producto: (i32, bool) = (42, true);

fn una_funcion_void() -> () {
    // Ejecuta un efecto secundario y retorna el producto
    // vac o ()
}

// Suma (con elementos) y Suma Vac a
enum Respuesta { Entero(i32), Booleano(bool) }
enum SumaVacía {} // Cardinalidad 0, equivale al tipo ! (
// never)

```

Si bien la suma vacía no aparece en el libro de Reynolds, más adelante daremos un resultado interesante.

Para entender las reglas de inferencia, primero debemos entender qué es un contexto de tipado (typing judgement). Este está dado por el contexto, denotado por π , e una expresión, θ es el tipo y su forma es la siguiente:

$$\pi \vdash e : \theta$$

Se lee como “ e tiene tipo θ bajo el contexto π ”.

El contexto es una lista que asigna tipo a los patrones (que en los casos más simples son variables).

Ejemplos de typing judgement válidos

- Con contexto:

$$\begin{aligned}
 m : \text{int}, n : \text{int} &\vdash m + n : \text{int} \\
 f : \text{int} \rightarrow \text{int}, x : \text{int} &\vdash f(fx) : \text{int} \\
 f : \text{int} \rightarrow \text{int} &\vdash \lambda x. f(fx) : \text{int} \rightarrow \text{int} \\
 &\vdash \lambda f. \lambda x. f(fx) : (\text{int} \rightarrow \text{int}) \rightarrow \text{int} \rightarrow \text{int}
 \end{aligned}$$

- Sin contexto:

$$\begin{aligned}
 \text{FACT} &: \text{int} \rightarrow \text{int} \\
 \text{ITERATE} &: \text{int} \rightarrow (\text{int} \rightarrow \text{int}) \rightarrow \text{int} \rightarrow \text{int} \\
 \text{ITERATE} &: \text{int} \rightarrow (\text{bool} \rightarrow \text{bool}) \rightarrow \text{bool} \rightarrow \text{bool}
 \end{aligned}$$

donde

$$\text{letrec fact} \equiv \lambda n. \text{if } n = 0 \text{ then } 1 \text{ else } n \times \text{fact}(n - 1)$$

$$\text{letrec iterate} \equiv \lambda n. \lambda f. \lambda x. \text{if } n = 0 \text{ then } x \text{ else } f(\text{iterate}(n - 1)f x)$$

2. Reglas de inferencia de tipos (TY RULES)

Abajo vamos a explicar algunas reglas de inferencia y su relación con algunos lenguajes de programación.

TY RULE: Variables

$$\pi \vdash v : \theta \quad \text{cuando } \pi \text{ contiene } v : \theta, \quad (15.3)$$

TY RULE: $\rightarrow$ Introduction

$$\frac{\pi, p : \theta \vdash e : \theta'}{\pi \vdash \lambda p. e : \theta \rightarrow \theta'} \quad (15.4)$$

TY RULE: $\rightarrow$ Elimination

$$\frac{\pi \vdash e_0 : \theta \rightarrow \theta' \quad \pi \vdash e_1 : \theta}{\pi \vdash e_0 e_1 : \theta'} \quad (15.5)$$

TY RULE: Product Introduction

$$\frac{\pi \vdash e_0 : \theta_0 \quad \dots \quad \pi \vdash e_{n-1} : \theta_{n-1}}{\pi \vdash \langle e_0, \dots, e_{n-1} \rangle : \mathbf{prod}(\theta_0, \dots, \theta_{n-1})}, \quad (15.6)$$

TY RULE: Product Elimination

$$\frac{\pi \vdash e : \mathbf{prod}(\theta_0, \dots, \theta_{n-1})}{\pi \vdash e.k : \theta_k} \quad \text{cuando } k < n, \quad (15.7)$$

Creemos que son bastante simples de entender, por lo que directamente les dejamos un ejemplo en Rust de la 15.6 y 15.7.

```
let x: i32 = 33;
let y: bool = true;
let tupla = (x, y);
let otra_tupla = (3, 2, true);

println!(
    "El tipo de 'tupla' es: {}",
    std::any::type_name_of_val(&tupla)
);
//El tipo de 'tupla' es: (i32, bool)
println!(
    "El tipo de 'otra_tupla.1' es: {}",
    std::any::type_name_of_val(&otra_tupla.1)
);
//El tipo de 'otra_tupla.1' es: i32
```

TY RULE: Sum Introduction

$$\frac{\pi \vdash e : \theta_k}{\pi \vdash @k e : \mathbf{sum}(\theta_0, \dots, \theta_{n-1})} \quad \text{cuando } k < n, \quad (15.8)$$

TY RULE: Sum Elimination

$$\frac{\begin{array}{l} \pi \vdash e : \mathbf{sum}(\theta_0, \dots, \theta_{n-1}) \\ \pi \vdash e_0 : \theta_0 \rightarrow \theta \\ \dots \\ \pi \vdash e_{n-1} : \theta_{n-1} \rightarrow \theta \end{array}}{\pi \vdash \mathbf{sumcase } e \text{ of } (e_0, \dots, e_{n-1}) : \theta}, \quad (15.9)$$

Como dijimos en la introducción un **sum** es equivalente a un enum, por lo que nos podemos imaginar el siguiente código si tenemos **sum**(*int*, *bool*)

```
// Definimos nuestro Tipo Suma con dos variantes (0 y 1)
enum Respuesta {
    Numerica(i32), // Variante @0
    Valor(bool), // Variante @1
}
```

Por lo que el equivalente a introducción @0 33 y a **sumcase** *e of* (*f*₀, *f*₁) sería

```
let e = Respuesta::Numerica(33);

match e {
    Respuesta::Numerica(v) => { // ejecucion de f_0
    },
    Respuesta::Valor(v) => { // ejecucion de f_1
    }
}
```

Algo que nos pareció interesante rescatar es el uso de **sum**() como la suma vacía, la cual en Rust sería un enum vacío, y permite dar a luz al **empty** type que en Rust se denota con ! y que de alguna forma denota nuestro $\perp$. Es decir que **unit** expresa la ausencia de información mientras que **empty** representa la imposibilidad de retornar información.

El lenguaje Rust posee un modelo formal que respalda la solidez de su sistema de tipos. El trabajo *RustBelt* (Jung et al., POPL 2018) proporciona una interpretación semántica para los tipos producto (véase su Sección 4.2), lo cual fundamenta el comportamiento de las tuplas en Rust. Por ello, utilizamos Rust para ilustrar estas reglas[2].

Ejemplo

Uso de las reglas de inferencia en la prueba de un *typing judgement*. Se nos pide demostrar lo siguiente:

$$\vdash \lambda f. \lambda x. f(f x) : (\mathbf{int} \rightarrow \mathbf{int}) \rightarrow \mathbf{int} \rightarrow \mathbf{int}$$

1. Lo de más afuera es un lambda, así que la última regla tiene que ser **introducción (15.4)**. Nos queda $f: \mathbf{int} \rightarrow \mathbf{int} \vdash \lambda x. f(fx): \mathbf{int} \rightarrow \mathbf{int}$.
2. Una vez más, se observa que lo aplicado anteriormente fue una introducción. Nos queda $f: \mathbf{int} \rightarrow \mathbf{int}, x: \mathbf{int} \vdash f(fx): \mathbf{int}$.
3. $f(fx)$ es una aplicación, entonces ¿qué aplicamos antes? **Eliminación (15.5)**. Necesitamos $f: \theta \rightarrow \mathbf{int}$ y $fx: \theta$. Me fijo en el contexto: para que se cumpla, f tiene que ser $\mathbf{int} \rightarrow \mathbf{int}$. Luego $fx: \mathbf{int}$ (que no está en el contexto, así que no es axioma).
4. $fx: \mathbf{int}$ es de nuevo una aplicación. Necesitamos $f: \theta \rightarrow \mathbf{int}$ y $x: \theta$. Me fijo en el contexto y, por último, descubro que $f: \mathbf{int} \rightarrow \mathbf{int}$ y $x: \mathbf{int}$.
5. f y x son variables $\rightarrow$ regla 15.3 (axiomas), y están en el contexto.

Los pasos 1–5 son cómo *encontramos* la prueba, buscándola desde la meta hacia los axiomas. La prueba final se escribe en el orden que usa el libro (de los axiomas hacia la meta): es la misma derivación recorrida al revés, donde los renglones 1 a 6 son justo los juicios que fuimos encontrando, pero en sentido inverso (las hojas, regla 15.3, primero; el juicio objetivo al final).

- | | |
|--|--------------|
| 1. $f: \mathbf{int} \rightarrow \mathbf{int}, x: \mathbf{int} \vdash f: \mathbf{int} \rightarrow \mathbf{int}$ | (15,3) |
| 2. $f: \mathbf{int} \rightarrow \mathbf{int}, x: \mathbf{int} \vdash x: \mathbf{int}$ | (15,3) |
| 3. $f: \mathbf{int} \rightarrow \mathbf{int}, x: \mathbf{int} \vdash fx: \mathbf{int}$ | (15,5, 1, 2) |
| 4. $f: \mathbf{int} \rightarrow \mathbf{int}, x: \mathbf{int} \vdash f(fx): \mathbf{int}$ | (15,5, 1, 3) |
| 5. $f: \mathbf{int} \rightarrow \mathbf{int} \vdash \lambda x. f(fx): \mathbf{int} \rightarrow \mathbf{int}$ | (15,4, 4) |
| 6. $\vdash \lambda f. \lambda x. f(fx): (\mathbf{int} \rightarrow \mathbf{int}) \rightarrow \mathbf{int} \rightarrow \mathbf{int}$ | (15,4, 5) |

Referencias

1. John C. Reynolds. *Theories of Programming Languages*. Cambridge University Press, 1998.
2. RustBelt: *Securing the Foundations of the Rust Programming Language*.